

Constraint on three people per spot

This constraint can clearly be ignored. Once we decide who goes where, we can send one person at a time. While that person is moving, everyone is either standing on their original spot or on their final spot, so the each spot contains at most two static people + maybe the person who's currently moving.

Exercise: would the solution still remain the same if the constraint said "at most two" instead of "at most three"? :)

Finding the shortest period (length)

For now, let's ignore the questions about specific letter order and let's start by figuring out the smallest possible regularity of a string we can assemble from a given multiset of n letters.

Let's start with an easy special case. What happens if the period length p divides n ? The whole string then consists of $r = n/p$ repetitions of the period. Clearly, this is possible if and only if the number of occurrences of each letter in our multiset is a multiple of r .

The same thing seen from the other side: the smallest possible p corresponds to the largest possible r , and the largest possible r is clearly the greatest common divisor of all letter counts.

Now let's generalize. We want to find the shortest period length p such that p does not *necessarily* divide n . In the general situation, the whole string consists of $r = \lfloor n/p \rfloor$ repetitions of the whole period, and at the end this is followed by one additional s -character prefix of the period, where $s = n - rp = n \bmod p$. (The special case we discussed above corresponds to $s = 0$: the extra prefix at the end is empty.)

For short, we will use "prefix of the period" for its first s letters and "suffix of the period" for its last $p - s$ letters.

Each character in the prefix of the period has $r + 1$ appearances in the whole string. Each character in the suffix of the period has just r appearances in the whole string.

Consider any one specific character of the alphabet, e.g., 'a'. Suppose it has a total of x appearances in our multiset.

We have to place some 'a's into the prefix and some 'a's into the suffix in such a way that the total number of 'a's in the resulting string will be x .

Let y be the number of 'a's we put in the prefix and z the number of 'a's we put in the suffix. We get the following equation: $x = (r + 1)y + rz$.

We now claim that **if p is minimal**, the values y and z are uniquely determined by x and p .

Why is that?

First, in the optimal string there cannot be any letter of the alphabet with $r + 1$ (or more) occurrences in the suffix of the period.

Why? Because if we had them, we could remove $r + 1$ copies of that letter from the suffix and instead place r copies of the letter into the prefix. The new period would also produce a string of length n using the exact same multiset of letters, and the new period is one shorter than the old one.

Thus, if somebody told you the optimal p , you could correspond the corresponding r and then conclude that for each letter we have at most r copies of it in the period's suffix.

This actually enforces how the copies of each letter have to be split between the prefix and the suffix of the period.

Consider both sides of the equation $x = (r + 1)y + rz$ modulo $r + 1$. We get $x \equiv rz \equiv (-1)z$, hence $z \equiv (-x)$, hence $z = (-x) \bmod (r + 1)$.

Note that in general this z can give us a negative value of y , in which case for this particular p there is no way to split the copies of the letter we are considering.

Armed with this knowledge we can iterate over all p until we find the smallest one where everything fits (no negative y values, sum of all y s and z s matches p).

This way we have obtained not just the shortest period, but the (as we now know, unique) split of that period into the prefix and the suffix.

What remains

We still have a few degrees of freedom. First, both within the prefix and within the suffix we can pick any order of letters. And second, we can pick any position for the leftmost letter of the resulting string (remember that it may be offset from the starting position).

Once we do make these choices, we'll have one candidate solution. That solution then still needs to be evaluated: how many steps does it take to reach it from the starting configuration?

Counting steps in general (for any permutation of letters)

Suppose you already chose which specific string you want to produce and also where. How can we count the minimal number of steps to do so?

Pick any letter of the alphabet, say 'a'. It has x copies in the source string and also x copies in the target string. We now claim that it is always optimal to send the leftmost 'a' in the source string to the leftmost 'a' in the target string, second 'a' to second 'a', and so on.

Once we realize this, it's easy to prove. Take any optimal solution. If some pair of 'a's swaps places (i.e., the 'a' that started farther right ends farther left), there had to be a moment during the solution where the two 'a's shared the same location. And at that moment we can modify the solution by swapping the destinations for the two 'a's: the left one will now continue to the destination that's farther left. Such a swap doesn't change the total number of steps, so the new solution is also optimal. If we do this each time two 'a's happen to be in the same location, we will obtain an optimal solution with no inversions.

Counting steps for a periodic string

Suppose we already found the optimal p and we know the two multisets of letters that form the prefix and the suffix of the period, respectively.

Now we have picked the location for the target string: the n consecutive positions where the letters should go.

Let's take p colors: first s dark shades of red and then $p - s$ dark shades of blue. Now let's go through the target locations and use these colors in a cyclic order to color them. (I.e., the first s locations will get

the different shades of red, the next $p - s$ locations will get shades of blue, then we repeat the red shades, and so on.)

Note that each specific color appears every p steps, so all locations that share the same color will, in the target string, share the same letter — at this moment we just don't know which one. Also note that shades of red correspond to the prefix and shades of blue to the suffix of the period.

Now let's look at the source string and pick any letter of the alphabet. We know the number y of times it appears in the prefix and the number z of times it appears in the suffix of the period. We will now take y *light* shades of red and z *light* shades of blue. We'll go through the source string left to right, and each time we see our letter, we give it the next color in this cyclic order. (I.e., the first y will be red, then z blue, then y red, and so on.)

Remember that we already argued that there is always an optimal solution in which the relative order of occurrences of the same letter does not change, we can now conclude that copies of the same letter that got the same shade will form one "equivalence class": they all correspond to the same letter in the period, i.e., in the target string they will be exactly p steps apart. The red equivalence classes will be parts of the prefix of the period, the blue ones will be in the suffix.

Once we finish going through the whole alphabet, we will have a total of s light-red equivalence classes and a total of $p - s$ light-blue equivalence classes. What remains is to find the optimal assignment: Which light-red class should go to which set of dark-red target locations? And once we know that, we'll have the exact same question for the blues.

And yes, we already used the magic word *assignment*. For each group of letters and each group of target locations we can easily calculate the cost of getting these letters to those locations. This will give us a matrix of costs, and our optimal solution is the optimal solution to the associated assignment problem — in other words, the minimum cost perfect matching between letter equivalence classes and their target locations.